



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



GUÍA PRÁCTICA SEMANA 07

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad I: Desarrollo Web Frontend

Tema: React Hooks: useState, useEffect, useContext, useRef, useReducer, useCallback, useMemo y Hooks Personalizados

Modalidad: Laboratorio

INTEGRANTES: _____

OBJETIVO DE LA PRÁCTICA

Deben desarrollar un **Dashboard de Gestión con Estado Complejo** que cumpla:

1. [useReducer](#) para estado de lista (add, remove, filter, sort) + [useState](#) para UI local.
2. [useContext](#) para tema claro/oscuro o preferencias de usuario.
3. [useEffect](#) para sincronizar preferencias con [localStorage](#) y cleanup.
4. [useMemo](#) para lista filtrada/ordenada, [useCallback](#) para handlers de botones.
5. [useRef](#) para auto-focus en input o acceso a nodo DOM sin re-render.
6. Hook personalizado useLocalStorage o useDebounce reutilizable.

Roles Técnicos

Rol	Responsabilidad
Arquitecto de Estado	useReducer , acciones, reducer function, inmutabilidad
Ingeniero de Efectos & Contexto	useEffect , useContext , sync con storage, cleanup
Optimizador de Rendimiento	useMemo , useCallback , useRef , análisis Profiler
QA & Hook Validator	Reglas de Hooks, ESLint warnings, documentación técnica, custom hook

Entregable

- Repositorio GitHub o ZIP con: [package.json](#), [src/](#), [README.md](#)



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



- [README.md](#) debe incluir:
 - Diagrama de flujo de estado y ciclo de render
 - Justificación técnica de cada hook utilizado (¿por qué no useState simple?)
 - Capturas de React DevTools Profiler (commits, tiempo de render, componentes optimizados)

Rúbrica de Evaluación

Criterio	Peso
Uso correcto de useReducer + manejo de estado complejo	25%
useContext + useEffect (sync, cleanup, dependencias)	20%
useMemo / useCallback aplicados estratégicamente (sin overuse)	20%
useRef + Hook personalizado funcional y reutilizable	20%
Validación Profiler/ESLint, documentación y trabajo en equipo	15%



CÓDIGO BASE DE LA GUÍA PRÁCTICA (Refactorizar)

src/contexts/ThemeContext.jsx

jsx

```
1 import { createContext, useState, useEffect } from 'react'
2
3 export const ThemeContext = createContext()
4
5 export const ThemeProvider = ({ children }) => {
6   // TODO: Inicializar estado con localStorage o valor por defecto
7   const [theme, setTheme] = useState('light')
8
9   // TODO: useEffect para aplicar clase al <body> y persistir en localStorage
10  // TODO: Función toggleTheme expuesta via value
11
12  return (
13    <ThemeContext.Provider value={{ theme, toggleTheme }}>
14      {children}
15    </ThemeContext.Provider>
16  )
17 }
```

src/hooks/useTasksReducer.js

js

```
1 // TODO: Definir tipos de acción: ADD, REMOVE, FILTER, SORT
2 const initialState = { items: [], filter: 'all', sort: 'asc' }
3
4 function tasksReducer(state, action) {
5   switch (action.type) {
6     // TODO: Implementar casos inmutables
7     default: return state
8   }
9 }
10
11 export default tasksReducer
```



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



src/hooks/useLocalStorage.js (Hook Personalizado)

```
js

1 import { useState, useEffect } from 'react'
2
3 // TODO: Implementar hook que sincronice estado con localStorage
4 // Recibe key y defaultValue, retorna [value, setValue]
5 export const useLocalStorage = (key, defaultValue) => {
6   // ...
7 }
```

src/components/TaskList.jsx

```
jsx

1 import { useMemo, useCallback, useRef } from 'react'
2
3 const TaskList = ({ tasks, onRemove, onToggle }) => {
4   const inputRef = useRef(null)
5
6   // TODO: useMemo para filtrar/ordenar tareas sin recalcular en cada render
7   const processedTasks = useMemo(() => {
8     // ...
9   }, [tasks])
10
11   // TODO: useCallback para estabilizar handlers pasados a hijos
12   const handleRemove = useCallback((id) => {
13     onRemove(id)
14   }, [onRemove])
15
16   // TODO: useEffect para focus al input cuando se añade tarea
17   // ...
18
19   return (
20     <ul>
21       {processedTasks.map(task => (
22         <li key={task.id}>
23           <span>{task.title}</span>
24           <button onClick={() => handleRemove(task.id)}>Eliminar</button>
25         </li>
26       ))}
27       <input ref={inputRef} placeholder="Nueva tarea..." />
28     </ul>
29   )
30 }
31
32 export default TaskList
```



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



src/App.jsx

```
jsx

1 import { useReducer, useContext } from 'react'
2 import tasksReducer from '../hooks/useTasksReducer'
3 import { ThemeProvider, ThemeContext } from '../contexts/ThemeContext'
4 import TaskList from '../components/TaskList'
5
6 const Dashboard = () => {
7   const [state, dispatch] = useReducer(tasksReducer, { items: [], filter: 'all', sort: 'asc' })
8   const { theme, toggleTheme } = useContext(ThemeContext)
9
10  // TODO: Conectar dispatch a handlers, renderizar TaskList y controles
11  return (
12    <div className={`dashboard ${theme}`}>
13      <button onClick={toggleTheme}>Cambiar Tema</button>
14      <h1>Gestor de Tareas</h1>
15      { /* TODO: Renderizar lista y formularios con dispatch */ }
16    </div>
17  )
18 }
19
20 export default function App() {
21   return (
22     <ThemeProvider>
23       <Dashboard />
24     </ThemeProvider>
25   )
26 }
```

NOTAS TÉCNICAS CLAVE

Concepto	Regla / Aplicación
Reglas de Hooks	Solo en nivel superior, nunca en loops/condiciones/funciones anidadas. Solo en componentes funcionales o custom hooks.
useEffect	Array de dependencias determina cuándo se ejecuta. Cleanup (return () => {}) evita memory leaks y race conditions.
useContext	Ideal para evitar prop drilling, pero no para estado que cambia frecuentemente (dispara re-render en todos los consumidores).
useReducer	Preferible a useState cuando el estado es complejo, tiene múltiples sub-valores o la lógica de actualización depende del estado anterior.



UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ
FACULTAD DE INGENIERÍA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS



useMemo / useCallback	Solo optimizar si hay bottleneck medible. Memoizar objetos/funciones pasados como props a hijos React.memo.
useRef	Persiste entre renders sin disparar re-render. Ideal para acceso DOM, timers, o valores mutables que no afectan UI.
Profiler	React DevTools → Profiler → Record. Analiza render time, commits, why did this render?. No optimizar sin medir.

VALIDACIÓN FINAL

1. Ejecutar npm run dev y verificar cero warnings de react-hooks/exhaustive-deps en ESLint.
2. Abrir React DevTools Profiler: grabar interacción, identificar componentes que se re-renderizan y validar que useMemo/useCallback reducen commits.
3. Verificar que useEffect tenga cleanup activo (simular navegación rápida o desmontaje).
4. Completar README.md con diagrama de estado, métricas Profiler y justificación técnica de hooks.
5. Subir a GitHub así mismo subir en rar su proyecto en el ADESA.